1. Why this matters in Spark

- Most engineers write Spark code.
- Very few understand how Spark executes it.
- In production systems:

  ➜ Jobs fail due to excessive shuffles

  ➜ Joins run 5× slower than expected

  ➜ Clusters are scaled unnecessarily

  ➜ Costs increase without clear reason
- The root cause?

  👉 Not reading the execution plan

Why Execution Plan Matters?

Your code:

```
df.join(customers, "customer_id") \
  .groupBy("country") \
  .count()
```

- Looks simple.

- But Spark internally generates:
  ➜ Logical plan
  ➜ Optimized logical plan
  ➜ Physical plan
  ➜ Stages
  ➜ Tasks

- If you don't read the plan, you don't know:
  ➜ How many shuffles happen
  ➜ Whether broadcast is used
  ➜ If sort-merge join is triggered
  ➜ If filters are pushed down

How to View Execution Plan

- ➜ **df.explain(True)**

- This shows:
  1. Parsed Logical Plan
  2. Analyzed Logical Plan
  3. Optimized Logical Plan
  4. Physical Plan

- Most engineers stop at writing code.
- Senior engineers read the **physical plan**.

Key Things to Look For in Physical Plan

---

1. **Exchange (Shuffle Indicator)**

- If you see:
  → **Exchange hashpartitioning(...)**

- That means shuffle is happening.
- Shuffle = network + disk I/O
- Too many exchanges = slow job.

2. **Join Type:**

- Look for:
  → BroadcastHashJoin (Good for small table joins)
  → SortMergeJoin (Heavy shuffle)
  →ShuffledHashJoin

- If small table is not broadcasted → performance opportunity.

3. **WholeStageCodegen:**

- If present:
  → ***(1) HashAggregate**

- Spark is optimizing using whole-stage code generation.
- Good sign.

Real Production Scenario

➜ Two similar queries:

- **Query A:**
  ➜ Uses broadcast
  ➜ 1 shuffle

- **Query B:**
  ➜ Uses sort-merge join
  ➜ 3 shuffles

- Code looks almost identical.
- Execution time:
  ➜ Query A → 4 minutes
  ➜ Query B → 28 minutes

- Difference?
  ➜ Execution plan awareness.

Spark Optimization Techniques Based on Plan

- **Enable Broadcast (if safe)**

```
from pyspark.sql.functions import broadcast

df.join(broadcast(customers), "customer_id")
```

- **Enable AQE (if not already)**

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

- AQE can:
  ➜ Switch join strategies
  ➜ Optimize shuffle partitions
  ➜ Handle skew automatically

Real Production Debugging Workflow

- When a Spark job is slow:

  ➜ Step 1: Run explain(True)
  ➜ Step 2: Count number of shuffles
  ➜ Step 3: Check join types
  ➜ Step 4: Inspect partition count
  ➜ Step 5: Review Spark UI (Stages & Tasks)

- Senior engineers:
  ➜ Never guess
  ➜ Always inspect execution plan

- Execution plan review prevents 70% of performance surprises.

Core Lesson

- Spark is not slow.
- Spark executes exactly what you ask it to.

- If you don’t understand the execution plan,
  you’re flying blind in distributed systems.

Interview & Production Insights

- Difference between logical and physical plan?
- What is shuffle and why is it expensive?
- How does Spark choose join strategy?
- What is AQE?

Final takeaway

- Spark is not slow.
- Spark executes exactly what your plan instructs.
- If you don't understand the execution plan, you're optimizing blindly.
- Mastering execution plans is the step from:

  ➔ Mid-level engineer → Senior Data Engineer.